

Full Stack Web Development





CHAPTER-2

RMI



RMI (Remote Method Invocation)

The **RMI** (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. The RMI allows an object to invoke methods on an object running in another JVM.

The RMI provides remote communication between the applications using two objects *stub* and *skeleton*.

Understanding stub and skeleton

RMI uses stub and skeleton object for communication with the remote object.

A **remote object** is an object whose method can be invoked from another JVM. Let's understand the stub and skeleton objects:



stub

The stub is an object, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object, it does the following tasks:

1. It initiates a connection with remote Virtual Machine (JVM),
2. It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
3. It waits for the result
4. It reads (unmarshals) the return value or exception, and
5. It finally, returns the value to the caller.

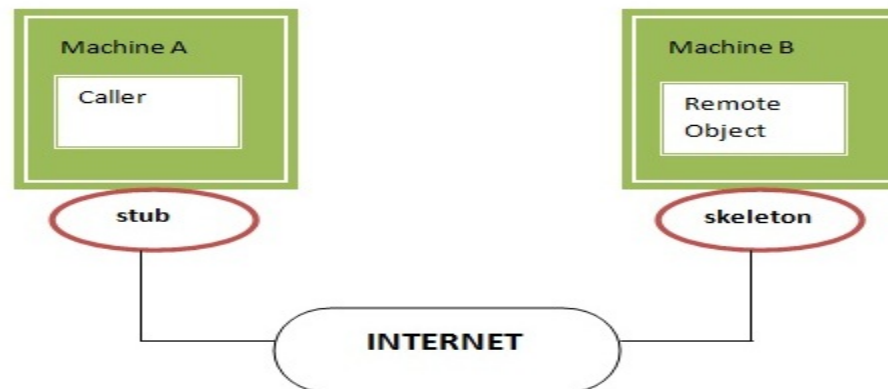


skeleton

The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:

1. It reads the parameter for the remote method
2. It invokes the method on the actual remote object, and
3. It writes and transmits (marshals) the result to the caller.

In the Java 2 SDK, an stub protocol was introduced that eliminates the need for skeletons.





Understanding requirements for the distributed applications

If any application performs these tasks, it can be distributed application.

1. The application need to locate the remote method
2. It need to provide the communication with the remote objects, and
3. The application need to load the class definitions for the objects.

The RMI application have all these features, so it is called the distributed application.



Java RMI Example

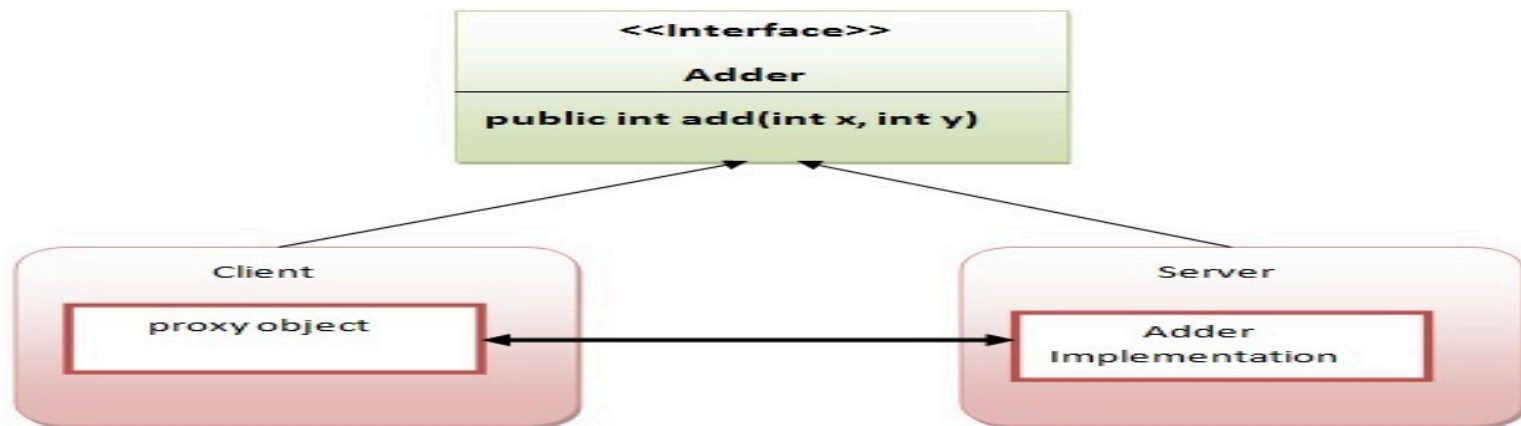
The is given the 6 steps to write the RMI program.

1. Create the remote interface
2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton objects using the rmic tool
4. Start the registry service by rmiregistry tool
5. Create and start the remote application
6. Create and start the client application



RMI Example

In this example, we have followed all the 6 steps to create and run the rmi application. The client application need only two files, remote interface and client application. In the rmi application, both client and server interacts with the remote interface. The client application invokes methods on the proxy object, RMI sends the request to the remote JVM. The return value is sent back to the proxy object and then to the client application.



1) create the remote interface

For creating the remote interface, extend the Remote interface and declare the RemoteException with all the methods of the remote interface. Here, we are creating a remote interface that extends the Remote interface. There is only one method named add() and it declares RemoteException.

```
import java.rmi.*;  
public interface Adder extends Remote{  
public int add(int x,  
int y)throws RemoteException;  
}
```



2) Provide the implementation of the remote interface

Now provide the implementation of the remote interface. For providing the implementation of the Remote interface, we need to

- Either extend the UnicastRemoteObject class,
- or use the exportObject() method of the UnicastRemoteObject class

In case, you extend the UnicastRemoteObject class, you must define a constructor that declares RemoteException.

```
import java.rmi.*;
import java.rmi.server.*;
public class AdderRemote extends UnicastRemoteObject implements
    Adder{
    AdderRemote()throws RemoteException{
    super();
    }
    public int add(int x,int y){return x+y;}
    }
```



3) create the stub and skeleton objects using the rmic tool.

Next step is to create stub and skeleton objects using the rmi compiler. The rmic tool invokes the RMI compiler and creates stub and skeleton objects.

```
rmic AdderRemote
```

4) Start the registry service by the rmiregistry tool

Now start the registry service by using the rmiregistry tool. If you don't specify the port number, it uses a default port number. In this example, we are using the port number 5000.

```
rmiregistry 5000
```



5) Create and run the server application

Now rmi services need to be hosted in a server process. The Naming class provides methods to get and store the remote object. The Naming class provides 5 methods.

```
public static java.rmi.Remote  
lookup(java.lang.String) throws java.  
rmi.NotBoundException, java.net.  
MalformedURLException, java.rmi.  
RemoteException;
```

It returns the reference of the remote object.

```
public static void bind(java.lang.String,  
java.rmi.Remote) throws java.rmi.  
AlreadyBoundException, java.net.  
MalformedURLException, java.rmi.  
RemoteException;
```

It binds the remote object with the given name.

CONTI.

```
public static void unbind(java.lang.String)
throws java.rmi.RemoteException, java.rmi.
NotBoundException, java.net.
MalformedURLException;
```

It destroys the remote object which is bound with the given name.

```
public static void rebind(java.lang.String, java.
rmi.Remote) throws java.rmi.RemoteException,
java.net.MalformedURLException;
```

It binds the remote object to the new name.

```
public static java.lang.String[] list(java.lang.
String) throws java.rmi.RemoteException, java.
net.MalformedURLException;
```

It returns an array of the names of the remote objects bound in the registry.



example, we are binding the remote object by the name sonoo.

```
import java.rmi.*;
import java.rmi.registry.*;
public class MyServer{
public static void main(String args[]){
try{
Adder stub=new AdderRemote();
Naming.rebind("rmi://localhost:5000/sonoo",stub);
}catch(Exception e){System.out.println(e);}
}
}
```



6) Create and run the client application

At the client we are getting the stub object by the lookup() method of the Naming class and invoking the method on this object. In this example, we are running the server and client applications, in the same machine so we are using localhost. If you want to access the remote object from another machine, change the localhost to the host name (or IP address) where the remote object is located.

```
import java.rmi.*;
public class MyClient{
public static void main(String args[]){
try{
    Adder stub=(Adder)Naming.lookup("rmi://localhost:5000/
    sonoo");
    System.out.println(stub.add(34,4));
catch(Exception e){}
}
}
```



For running **this** rmi example,

) compile all the java files

```
javac *.java
```

)create stub and skeleton object by rmic tool

```
rmic AdderRemote
```

)start rmi registry in one command prompt

```
rmiregistry 5000
```

)start the server in another command prompt

```
java MyServer
```

)start the client application in another command prompt

```
java MyClient
```



Meaningful example of RMI application with database

Consider a scenario, there are two applications running in different machines. Let's say MachineA and MachineB, machineA is located in United States and MachineB in India. MachineB want to get list of all the customers of MachineA application.

Let's develop the RMI application by following the steps.

1) Create the table

First of all, we need to create the table in the database. Here, we are using Oracle10 database.

CUSTOMER400

[Table](#) [Data](#) [Indexes](#) [Model](#) [Constraints](#) [Grants](#) [Statistics](#) [UI Defaults](#) [Triggers](#) [Dependencies](#) [SQL](#)

[Query](#) [Count Rows](#) [Insert Row](#)

EDIT	ACC_NO	FIRSTNAME	LASTNAME	EMAIL	AMOUNT
	67539876	James	Franklin	franklin1james@gmail.com	500000
	67534876	Ravi	Kumar	ravimalik@gmail.com	98000
	67579872	Vimal	Jaiswal	jaiswalvimal32@gmail.com	9380000
row(s) 1 - 3 of 3					

[Download](#)



2) Create Customer class and Remote interface

```
package com.javatpoint;  
public class Customer implements java.io.
```

```
Serializable{
```

```
    private int acc_no;
```

```
    private String firstname,lastname,email;
```

```
    private float amount;
```

Note: Customer class must be Serializable.

```
//getters and setters
```

```
}
```

```
package com.javatpoint;
```

```
import java.rmi.*;
```

```
import java.util.*;
```

```
interface Bank extends Remote{
```

```
    public List<Customer> getCustomers()throws RemoteExce  
ption;  
}
```



3) Create the class that provides the implementation of Remote interface

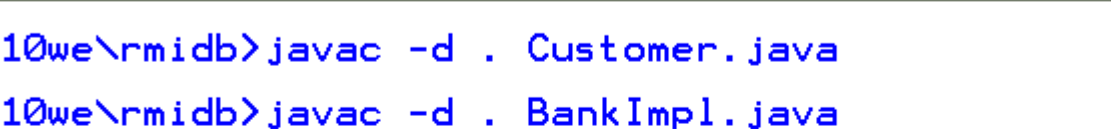
```
package com.javatpoint;
import java.rmi.*;
import java.rmi.server.*;
import java.sql.*;
import java.util.*;
class BankImpl extends UnicastRemoteObject implements Bank{
BankImpl()throws RemoteException{}

public List<Customer> getCustomers(){
List<Customer> list=new ArrayList<Customer>();
try{
Class.forName("oracle.jdbc.driver.OracleDriver");
Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe",
"system","oracle");
PreparedStatement ps=con.prepareStatement("select * from customer400");
ResultSet rs=ps.executeQuery();
```




```
while(rs.next()){  
    Customer c=new Customer();  
    c.setAcc_no(rs.getInt(1));  
    c.setFirstname(rs.getString(2));  
    c.setLastname(rs.getString(3));  
    c.setEmail(rs.getString(4));  
    c.setAmount(rs.getFloat(5));  
    list.add(c);  
}  
  
con.close();  
}catch(Exception e){System.out.println(e);}  
return list;  
}  
//end of getCustomers()  
}
```



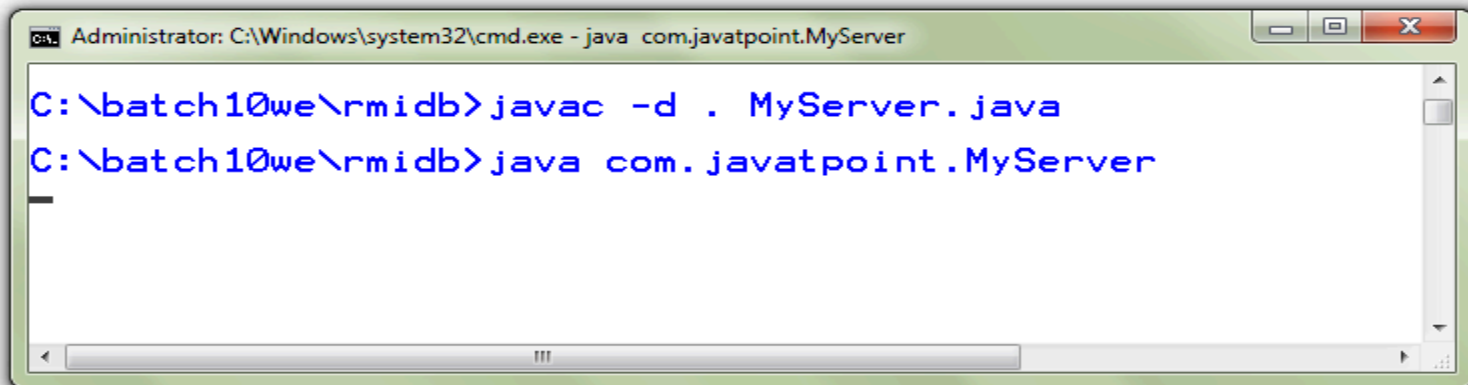


```
C:\batch10we\rmidb>javac -d . Customer.java
C:\batch10we\rmidb>javac -d . BankImpl.java
C:\batch10we\rmidb>rmic com.javatpoint.BankImpl
C:\batch10we\rmidb>rmiregistry 6666
```



5) Create and run the Server

```
package com.javatpoint;  
import java.rmi.*;  
public class MyServer{  
  public static void main(String args[])throws Exception{  
    Remote r=new BankImpl();  
    Naming.rebind("rmi://localhost:6666/javatpoint",r);  
  }  
}
```

A screenshot of a Windows Command Prompt window. The title bar reads "Administrator: C:\Windows\system32\cmd.exe - java com.javatpoint.MyServer". The command prompt shows the following commands and their output:
C:\batch10we\rmidb>javac -d . MyServer.java
C:\batch10we\rmidb>java com.javatpoint.MyServer
The window has a scrollbar on the right and a status bar at the bottom.

6) Create and run the Client

```
package com.javatpoint;  
import java.util.*;  
import java.rmi.*;  
public class MyClient{  
  public static void main(String args[])throws Exception{  
    Bank b=(Bank)Naming.lookup("rmi://localhost:6666/javatpoint");  
  
    List<Customer> list=b.getCustomers();  
    for(Customer c:list){  
      System.out.println(c.getAcc_no()+" "+c.getFirstname()+" "+c.  
        getLastName()  
        +" "+c.getEmail()+" "+c.getAmount());  
    }  
  
  }  
}
```





C:\Administrator: C:\Windows\system32\cmd.exe

```
C:\batch10we\rmidb>javac -d . MyClient.java
```

```
C:\batch10we\rmidb>java com.javatpoint.MyClient
```

```
67539876 James Franklin franklin1james@gmail.com 500000.0
```

```
67534876 Ravi Kumar ravimalik@gmail.com 98000.0
```

```
67579872 Vimal Jaishwal jaishwalvimal32@gmail.com 9380000.0
```

```
C:\batch10we\rmidb>
```



× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.

